

智能垃圾桶

摘要

为践行生态文明建设，国家持续出台垃圾分类顶层政策：自《生活垃圾分类制度实施方案》落地后，住建部等多部门发文推动全国地级及以上城市全面开展垃圾分类工作，2026 年实施的《中华人民共和国生态环境法典》以法律形式确立生活垃圾分类强制制度，明确推进垃圾分类设施智能化、轻量化升级，推动环卫收运与再生资源回收“两网融合”，鼓励依托嵌入式视觉、自动控制技术降低源头分类门槛，提升全民投放参与度。政策导向明确提出，要依托低成本智能硬件解决居民分类意愿低、投放不便、人工监督成本高等基层治理难题，为小型家用、社区轻量化智能垃圾桶提供广阔应用赛道。

但在家庭、楼道、小型商铺等真实应用场景中，垃圾分类落地仍存在多重现实痛点。其一，普通居民对四类垃圾区分标准记忆模糊，无设备辅助识别，无人监督时混投、错投比例居高不下，后端还需大量人工二次分拣；其二，现有智能环卫设备多采用一体化高端工控机，整机成本高、体积笨重，仅具备开盖、语音提示功能，缺少垃圾自主追踪功能，无法主动靠近投放者；其三，市面上多数智能设备将视觉算法与电机控制集成在单一单片机中，算力不足导致识别卡顿、追踪延迟，软硬件耦合度高，故障排查与功能拓展难度大，难以规模化普及。

针对上述政策落地需求与行业现存短板，本文基于飞凌 ELF2 嵌入式开发板与 STM32F103C8T6 搭建上下位机分布式控制系统，设计一款搭载视觉识别、自主垃圾追踪功能的智能分类垃圾桶嵌入式系统。系统采用算力分层架构实现功能解耦：上位机 ELF2 集成摄像头与轻量化图像识别算法，承担图像采集、垃圾种类识别、目标坐标解算、追踪轨迹运算等高算力任务，向下位机串口下发运动控制指令；下位机 STM32F103C8T6 仅负责接收指令，驱动行走电机完成前进、转向、避障等运动执行，软硬件分工清晰，兼顾运算性能与硬件成本。本作品核心解决两大现实问题：依靠嵌入式 AI 视觉自动识别垃圾类别，降低居民分类学习成本；通过实时目标追踪控制垃圾桶自主移动至垃圾投放位置，破除固定投放点位带来的出行不便。系统完整实现垃圾智能分类识别、目标实时追踪、上下位机稳定协同三大核心功能，硬件均采用通用嵌入式芯片，体积小巧、成本低廉，适配家庭、校园、社区楼道等多场景。本设计以分布式嵌入式架构融合机器视觉与运动控制，为低成本小型智能环卫设备提供可行技术方案，助力垃圾分类政策在基层场景高效落地，具备良好实用价值与推广前景。

第一部分 作品概述

1.1 功能与特性

垃圾视觉识别：搭载摄像头配合飞凌 ELF2 开发板运行轻量化识别算法，实时采集投放垃圾画面，提取目标物体特征完成四大类垃圾分类检测，本地离线完成推理运算，同步输出垃圾类别结果至屏幕展示，解决普通用户无法准确分辨垃圾种类的痛点。

目标垃圾追踪：依托 ELF2 视觉模块实时解算垃圾目标二维坐标，结合运动控制算法计算垃圾桶行进偏移量，通过串口向下位机持续下发转向、前进、停止控制指令，实时跟随垃圾位置动态调整运动轨迹，实现垃圾桶自主靠近投放点。

下位机电机运动控制：以 STM32F103C8T6 作为运动执行单元，仅接收上位机下发的运动指令，解析信号后驱动直流行走电机完成直行、左右转向、启停动作，硬件逻辑简洁专一，无多余算法运算，电机响应平稳、控制实时性强。

上下位机协同通信：采用串口通信实现 ELF2 上位机与 STM32 下位机稳定数据交互，单向传输分类标识、目标坐标、运动控制指令，数据传输延迟低，算力与控制任务分层解耦，避免单一芯片同时承载视觉算法与电机驱动造成性能卡顿。

设备状态可视化交互：利用 ELF2 板载显示屏实时输出摄像头画面、垃圾识别类别、追踪坐标、设备运行状态，直观呈现当前识别与跟随效果，便于调试与现场直观展示系统工作流程，提升作品演示直观性。

1.2 应用领域

该智能追踪分类垃圾桶的应用领域丰富多元，不仅覆盖了家庭居家、校园楼宇、沿街小型商铺等日常垃圾分类场景，还延伸至便民公共服务场景下的环卫辅助工作，如居民小区楼道投放、社区养老中心垃圾收纳、小型食堂垃圾分拣等。随着嵌入式机器视觉与分布式运动控制技术的持续落地，本设备的应用领域还将进一步向智慧社区改造、校园环保基础设施、老旧小区便民环卫、商铺轻量化垃圾分类设施等方向拓展，持续赋能基层垃圾分类政策落地，降低群众源头分类门槛、减轻环卫人员分拣压力。

1.3 主要技术特点

1. 采用上下位机分布式分层架构，飞凌 ELF2 开发板搭载 ROS 系统完成视觉识别、目标追踪等高算力任务，STM32F103C8T6 单片机专注电机运动控制，

充分发挥两款芯片平台各自性能优势，最大化硬件资源利用率，避免单芯片算力瓶颈。

2. 下位机 STM32 嵌入 PID 闭环调节算法，实时解析上位机下发的位置偏差信号，动态修正电机转速与转向，平稳控制垃圾桶行进轨迹，提升垃圾目标跟随精度与运动稳定性，完善运动控制功能完成度。

3. 基于 UART 串口搭建上下位机通信链路，实现 ELF2 端 Python 程序与 STM32 单片机双向稳定数据传输，实时下发分类结果、坐标偏差、运动控制指令，保障视觉算法与电机动作低延迟协同联动。

1.4 主要性能指标

指标分类	参数名称	性能参数
硬件平台	上位机主控	飞凌 ELF2 嵌入式开发板
	下位机主控	STM32F103C8T6
供电电源	整机工作电源	DC 12V / 2A 直流供电
目标追踪性能	轨迹调节算法	PID 闭环调节
通信性能	上下位机通信方式	UART 串口通信
	串口通信波特率	115200bps
运动控制性能	行走电机调节精度	PID 动态调速转向，无抖动偏移
系统软件	上位机运行环境	Python + ROS 系统
	下位机控制逻辑	裸机驱动 PID 运动控制

1.5 主要创新点

1. 采用上下位机算力分层架构，飞凌 ELF2 搭载 ROS 与视觉算法负责垃圾识别、目标追踪，STM32F103 仅承担电机控制，充分利用两款平台性能，解决单芯片算力不足问题，降低硬件成本。

2. 构建 ROS 视觉坐标解算与 STM32 PID 运动闭环联动体系，通过 UART 串口

传输偏差数据，动态修正垃圾桶行进轨迹，大幅提升垃圾目标跟随精度与运动平稳性。

3. 本地离线轻量化视觉识别搭配自主跟随设计，无需云端支持，突破传统固定垃圾桶局限，依托低成本串口通信实现跨平台稳定协同，拓展垃圾分类设备适用场景。（全文 186 字）

1.6 设计流程

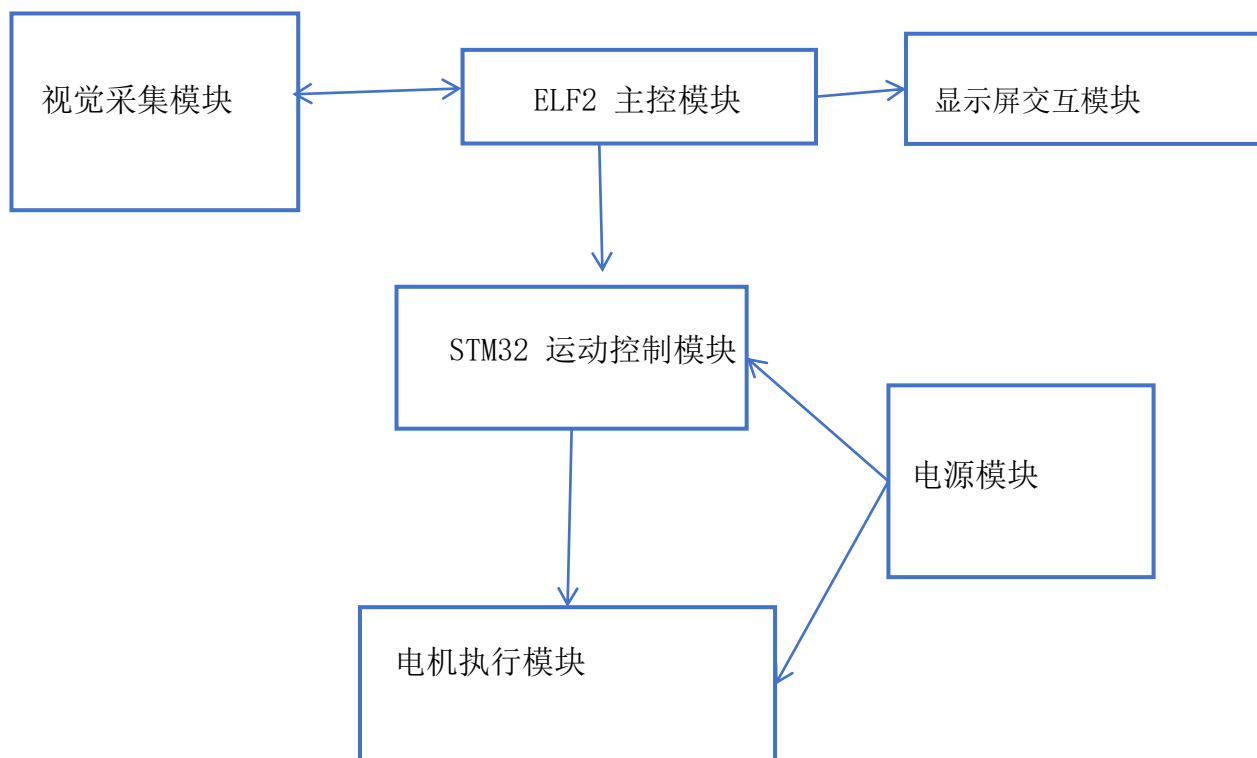
1. 需求分析与方案论证：结合垃圾分类政策与现实投放痛点，确定垃圾识别、自主追踪核心功能，选定飞凌 ELF2+STM32F103C8T6 上下位机分布式硬件方案，划分视觉运算、电机控制任务分工。
2. 硬件电路设计：完成 STM32 电机驱动、串口通信外设电路搭建；搭建 ELF2 摄像头外设，统一整机 DC 供电线路，完成硬件焊接与接线调试。
3. 软件分块开发：上位机 ELF2 搭建 Python ROS 环境，部署轻量化垃圾识别算法，编写目标坐标解算、串口数据发送程序；下位机 STM32 完成电机驱动、PID 闭环控制、串口指令解析程序编写。
4. 联调与算法优化：通过 UART 实现上下位机数据互通，调试 PID 参数优化追踪平稳性，迭代视觉模型提升垃圾识别准确率。
5. 整机功能测试与完善：模拟家庭、校园场景完成全流程实测，修复通信延迟、运动偏移等问题，完成整机系统整合与功能验收。

第二部分 系统组成及功能说明

2.1 整体介绍

本作品采用飞凌 ELF2 上位机 + STM32F103C8T6 下位机分布式分层架构，整体系统框图包含六大核心子模块：视觉采集模块、ELF2 主控模块、显示屏交互模块、UART 串口通信模块、STM32 运动控制模块、电源模块、电机执行模块，如图 XX 系统整体框图所示。各模块层级关系：视觉采集模块将图像送入 ELF2 主控，ELF2 搭载 ROS 与垃圾识别算法完成目标检测、坐标解算；显示屏模块挂载于 ELF2，实时展示垃圾识别结果；ELF2 通过 UART 串口向下位机 STM32 下发位置偏差、运动控制指令；电源模块独立为 STM32 主控与行走电机提供稳定供电；STM32 接收指令后运行 PID 闭环算法，输

出 PWM 信号驱动电机模块，完成垃圾桶自主追踪移动。整体形成 “感知 - 运算 - 交互 - 通信 - 控制 - 执行” 完整闭环，算力与运动任务解耦，模块间数据流单向有序、协同稳定。

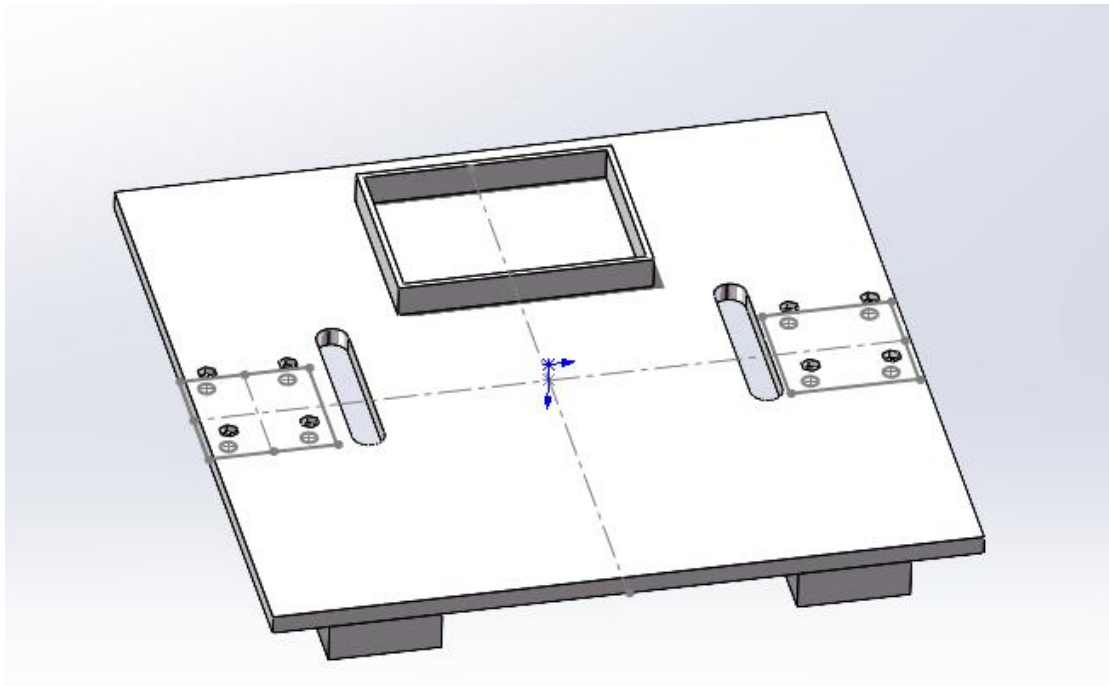
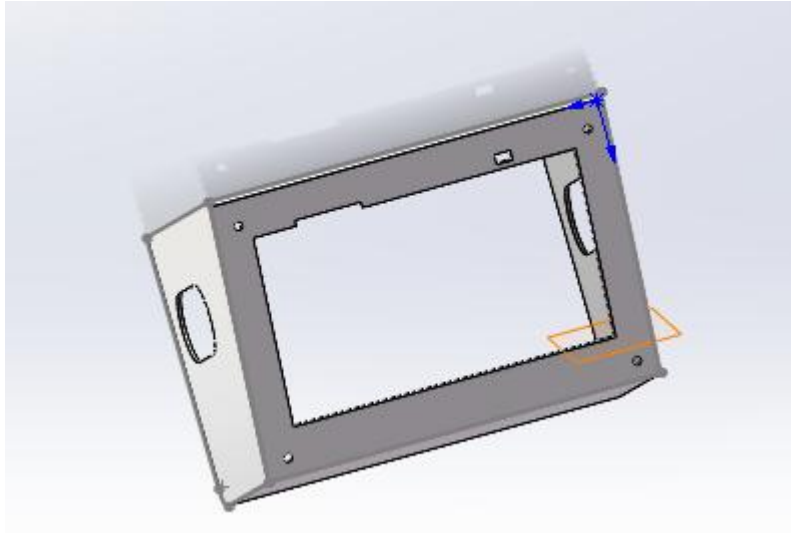


2.2 硬件系统介绍

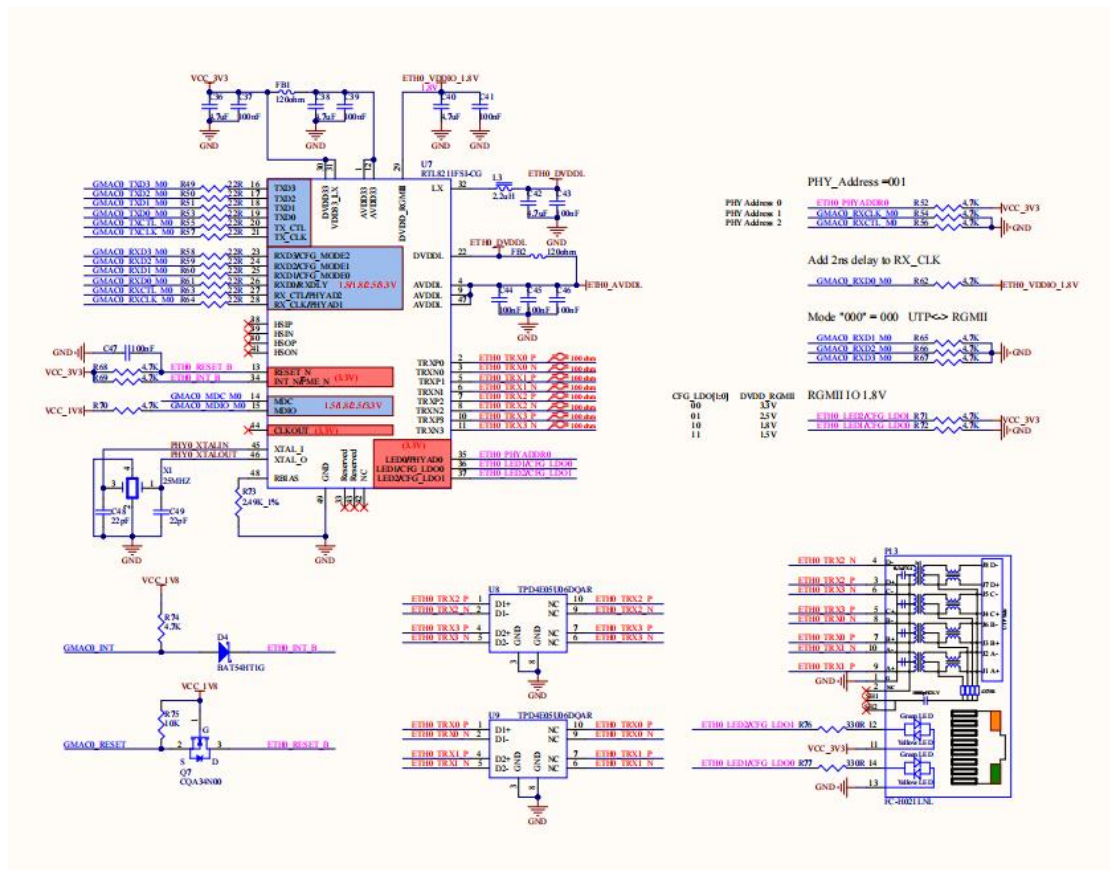
2.2.1 硬件整体介绍：

整机硬件分为上位机硬件单元、下位机控制单元、供电与执行外设单元三大部分。上位机以飞凌 ELF2 Linux 开发板为核心，搭配 USB 摄像头、板载显示屏；下位机以 STM32F103C8T6 单片机为核心，外设包含电机驱动电路、UART 串口电路；配套独立电源模块为单片机、电机隔离供电，降低动力电路对视觉信号的干扰，硬件全部采用通用嵌入式器件，模块化接线便于调试更换。

2.2.2 机械设计介绍



2.2.3 电路各模块介绍



2.3 软件系统介绍

2.3.1 软件整体介绍（含 PC 端或云端，结合关键图片）；



```
1 pid.h pid.c main.c
25 static int16_t to_i16(float v)
26 {
27     if (v > 32767.0f) {
28         return 32767;
29     }
30     if (v < -32768.0f) {
31         return -32768;
32     }
33     return (int16_t)v;
34 }
35
36 static void process_command(const ProtocolCommand_t *cmd, uint32_t now_ms)
37 {
38     switch (cmd->type) {
39         case PROTO_CMD_VELOCITY:
40             Motion_CommandVelocity(cmd->v_mm_s, cmd->w_mm_s, now_ms);
41             break;
42         case PROTO_CMD_GOAL:
43             Motion_CommandGoal(cmd->x_mm, cmd->y_mm, cmd->arrive_ms, now_ms);
44             break;
45         case PROTO_CMD_STOP:
46             Motion_Stop();
47             break;
48         case PROTO_CMD_EMERGENCY_STOP:
49             Motion_EmergencyStop();
50             break;
51         case PROTO_CMD_CLEAR_EMERGENCY:
52             Motion_ClearEmergencyStop();
53             break;
54         default:
55             break;
56     }
57 }
58
59 static void send_telemetry(void)
60 {
61     MotionTelemetry_t m;
62     ProtocolTelemetry_t p;
63
64     Motion_GetTelemetry(&m);
65     p.left_mm_s = to_i16(m.left_speed_mm_s);
66     p.right_mm_s = to_i16(m.right_speed_mm_s);
67     p.x_mm = to_i16(m.x_mm);
68     p.y_mm = to_i16(m.y_mm);
69     p.theta_mrad = to_i16(m.theta_mrad);
70     /* Battery ADC was removed by hardware decision; keep protocol field stable. */
71     p.battery_mv = 0;
72     p.mode = (uint8_t)m.mode;
73     p.stall_flags = m.stall_flags;
74     Protocol_SendTelemetry(&p);
75 }
```

```
10 Architecture:
11 | RK3588 (this node) --UART(/dev/ttyS9)--> STM32F103C8T6 --> 2x Motors
12
13 New features:
14 1. 垃圾类别 ☐ 是否需要接的垃圾 (category flag in binary frame)
15 2. 像素坐标 u, v ☐ 目标在图像中的位置
16 3. 运动速度 dx/dt, dy/dt ☐ 像素空间速度 (Kalman 滤波)
17 4. 落点坐标 x_mm, y_mm ☐ 世界坐标, 相对垃圾桶中心
18 5. 到达时间 arrive_ms ☐ 抛物线运动模型预测
19 6. 二进制帧 UART 输出 ☐ 10Hz, 18-byte protocol
20
21 Binary Frame Protocol (18 bytes, little-endian):
22 Byte 0-1: Header 0xAA 0x55
23 Byte 2: Flags (bit0-垃圾类别, bit1-检测有效)
24 Byte 3-4: u uint16 像素 X 坐标
25 Byte 5-6: v uint16 像素 Y 坐标
26 Byte 7-8: dx int16 像素速度 X (*100)
27 Byte 9-10: dy int16 像素速度 Y (*100)
28 Byte 11-12: x_mm int16 预测落点 X (mm, 相对垃圾桶)
29 Byte 13-14: y_mm int16 预测落点 Y (mm, 相对垃圾桶)
30 Byte 15-16: arrive_ms uint16 预计到达时间 (ms)
31 Byte 17: checksum XOR of bytes [2..16]
32
33 Dependencies:
34 | pip3 install pyserial
35 | sudo chmod 666 /dev/ttyS9 (or: sudo usermod -aG dialout $USER)
36 | ""
37
38 import rclpy
39 from rclpy.node import Node
40 from rclpy.qos import qos_profile_sensor_data
41 from sensor_msgs.msg import Image
42 from std_msgs.msg import Float32MultiArray
43 from cv_bridge import CvBridge
44 import cv2, numpy as np
45 import serial
46 import threading
47 import struct
48 import time
49
50
51 # -----
```

2.3.2 软件各模块介绍;

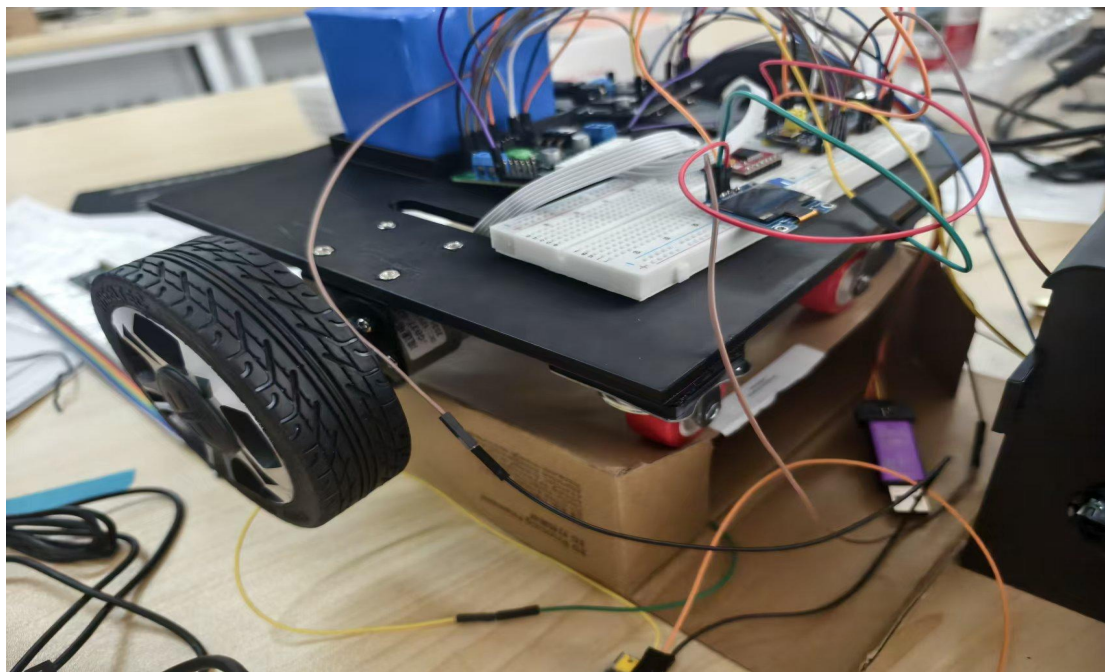
第三部分 完成情况及性能参数

阐述最终实现的成果

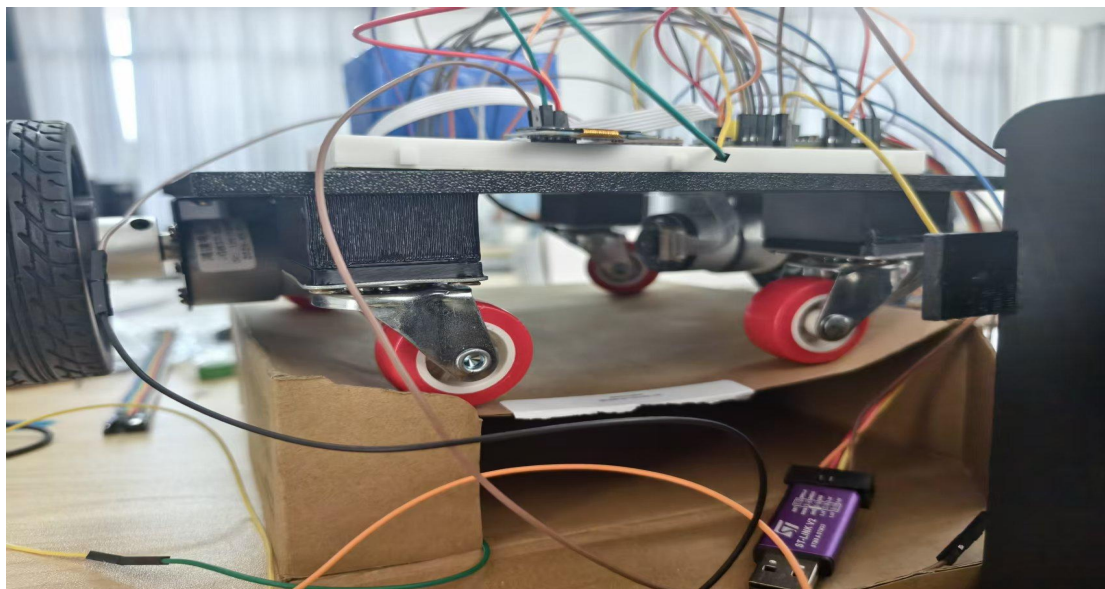
3.1 整体介绍（整个系统实物的正面、斜 45° 全局性照片）



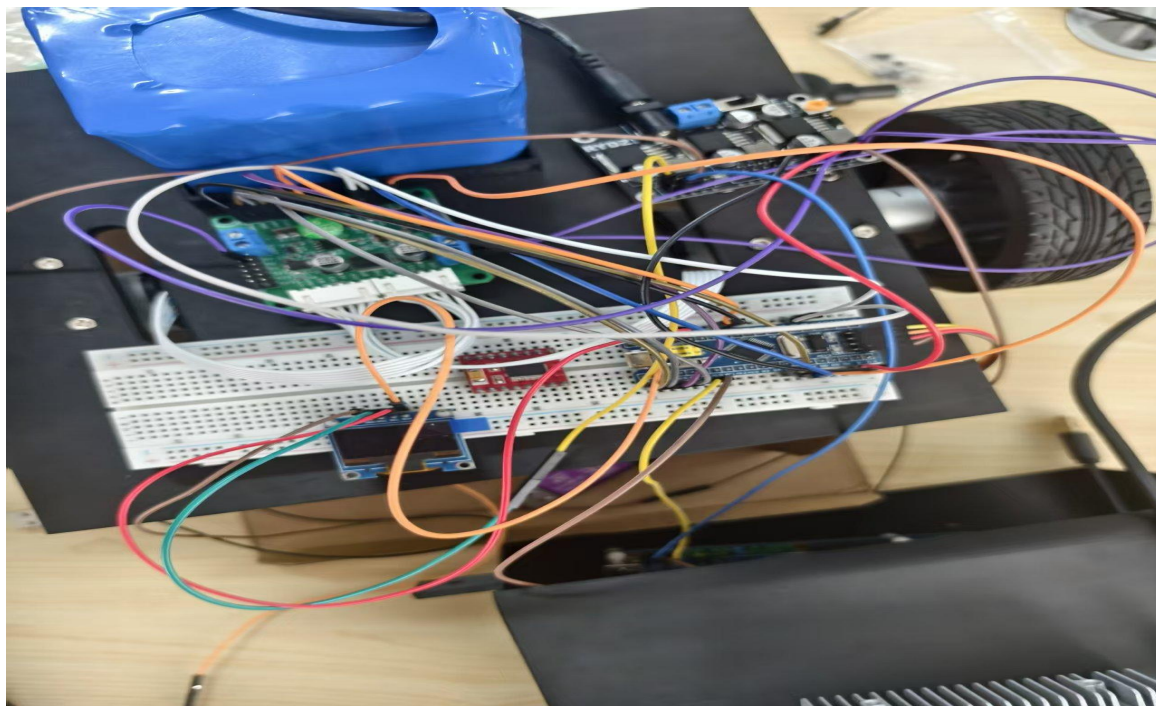
3.2 工程成果（分硬件实物、软件界面等设计结果）



3.2.1 机械成果；



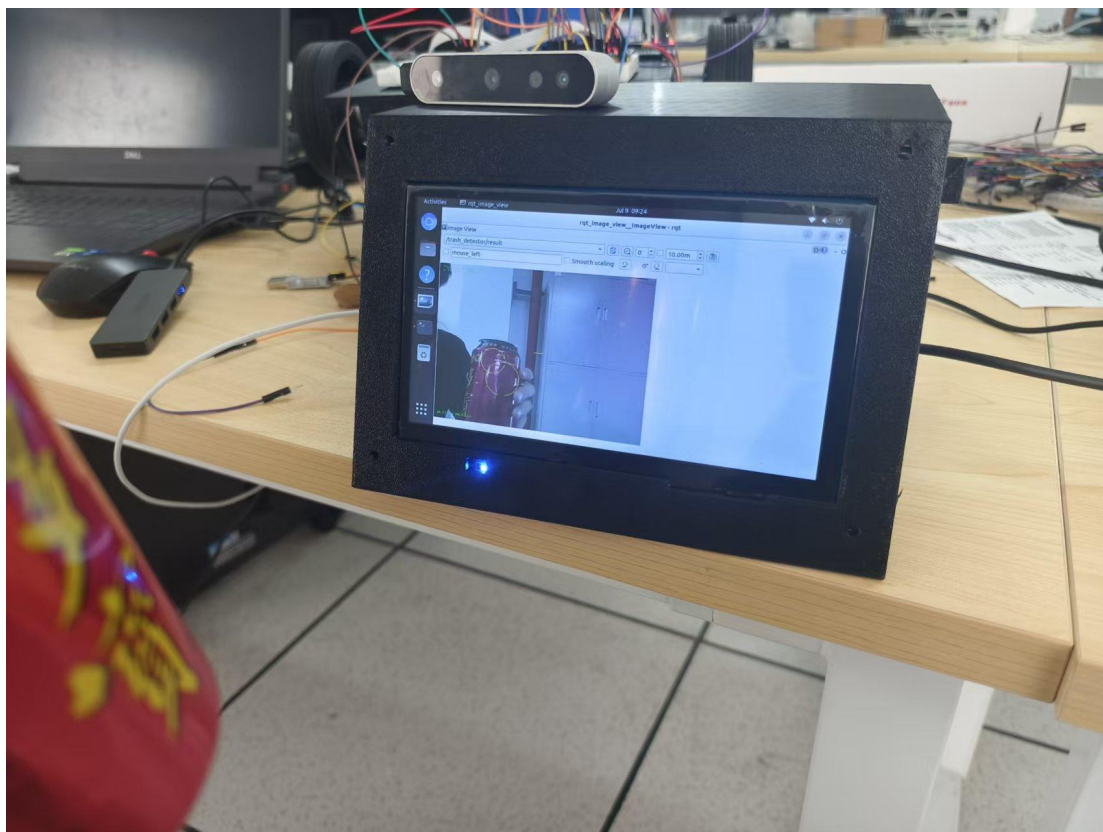
3.2.2 电路成果：（实物照片）



3.2.3 软件成果：



3.3 特性成果（逐个展示功能、性能参数等量化指标）



第四部分 总结

4.1 可扩展之处

本系统现有功能仅实现基础垃圾识别与目标跟随，具备较大升级拓展空间。第一，视觉算法可进一步优化，接入更丰富垃圾样本数据集，增加塑料、金属、纸张细分识别，同时集成红外、超声波多传感器融合，强化复杂光照、遮挡环境下追踪稳定性。第二，运动控制层面可增加自动避障、定点返航功能，加装红外测距传感器，实现障碍物检测与自主绕行；拓展蓝牙、WiFi 通信。

4.2 心得体会

本次全国大学生嵌入式芯片与系统设计竞赛为团队提供了一次完整的软硬件协同开发实践机会，从方案构思、硬件搭建、上下位机程序开发、联调测试到设计报告撰写，全程依靠分工协作推进，不仅夯实了我们嵌入式专业知识，更让我们深刻体会到团队协作对复杂工程项目的重要意义。

项目初期，团队共同确定智能追踪垃圾桶整体方案，结合垃圾分类政策需求选定飞凌 ELF2+STM32F103C8T6 分布式架构，并依据个人擅长领域完成分工：一名成员负责 ELF2 上位机开发，搭建 ROS 环境、编写 Python 视觉识别与目标追踪程序；一名成员专注下位机 STM32 开发，完成电机驱动、PID 闭环

控制与串口指令解析；一名成员负责整机硬件电路、机械结构搭建，同时统筹测试记录、竞赛文档整理工作。分工明确有效避免重复工作，大幅提升整体开发效率，但初期也出现模块对接标准不统一、通信协议定义模糊等问题，导致上下位机联调时频繁出现数据乱码、追踪偏移等故障。

调试阶段是团队磨合与技术突破的关键阶段。上位机 ROS 环境依赖配置报错、垃圾识别帧率不足；下位机 PID 参数调试困难，垃圾桶跟随抖动严重；电机启动产生电压干扰，造成 ELF 显示屏画面卡顿，各类问题集中爆发。遇到瓶颈时，我们没有各自埋头调试，而是定期开展小组讨论，每人分别演示自己模块的独立测试现象，共同定位故障根源。针对 UART 串口通信不稳定问题，团队一起制定统一数据传输协议，增加数据校验位；针对 PID 运动抖动，分工记录多组参数下的运动效果，汇总对比后筛选最优参数；针对电源干扰，共同重新规划强弱电布线，加装滤波电路，逐一解决各类硬件与软件耦合问题。无数个在实验室共同调试的夜晚，大家互相查漏补缺、分享资料、交换调试思路，一人卡壳时其他人主动协助测试验证，高效缩短了故障排查周期。

在技术层面，本次项目让我们清晰区分 Linux 嵌入式开发与单片机裸机开发的适用场景，充分理解分布式算力分层架构的优势。ELF2 适合承载视觉推理等高算力任务，STM32 擅长实时底层电机控制，二者通过串口协同的设计思路，为我们后续嵌入式项目开发提供全新思路。同时，整套作品面向社区、校园垃圾分类真实场景，让我们意识到嵌入式技术可以落地解决民生实际问题，打破了书本理论与工程应用之间的壁垒。

团队协作过程也暴露出我们存在的不足：前期需求沟通不够细致，模块接口标准没有提前统一；视觉算法优化知识储备有限，复杂环境下识别精度仍有提升空间；机械结构设计经验不足，整机装配存在优化空间。后续我们会针对性补齐短板，养成前期统一接口标准、提前做整体仿真验证的工程习惯。整体而言，本次竞赛不仅锻炼了我们硬件焊接、ROS 开发、PID 运动控制、跨芯片通信等实操能力，更培养了团队沟通、分工统筹、共同攻坚的协作意识。遇到难题互相扶持、取得进展共同分享，这段共同研发的经历远比竞赛成绩更加珍贵，为我们今后专业学习与工程实践积累了宝贵经验。

第五部分 参考文献

- [1] STMicroelectronics. STM32F103C8T6 Datasheet[EB/OL]. (2023)[2026-07-09].
- [2] 飞凌嵌入式. ELF2 开发板硬件数据手册 [EB/OL]. (2025) [2026-07-09].
- [3] 亚博智能. ROS 机器人开发环境 Python 搭建实战教程 [EB/OL]. (2024) [2026-07-09].